Data Structures and Algorithms I.

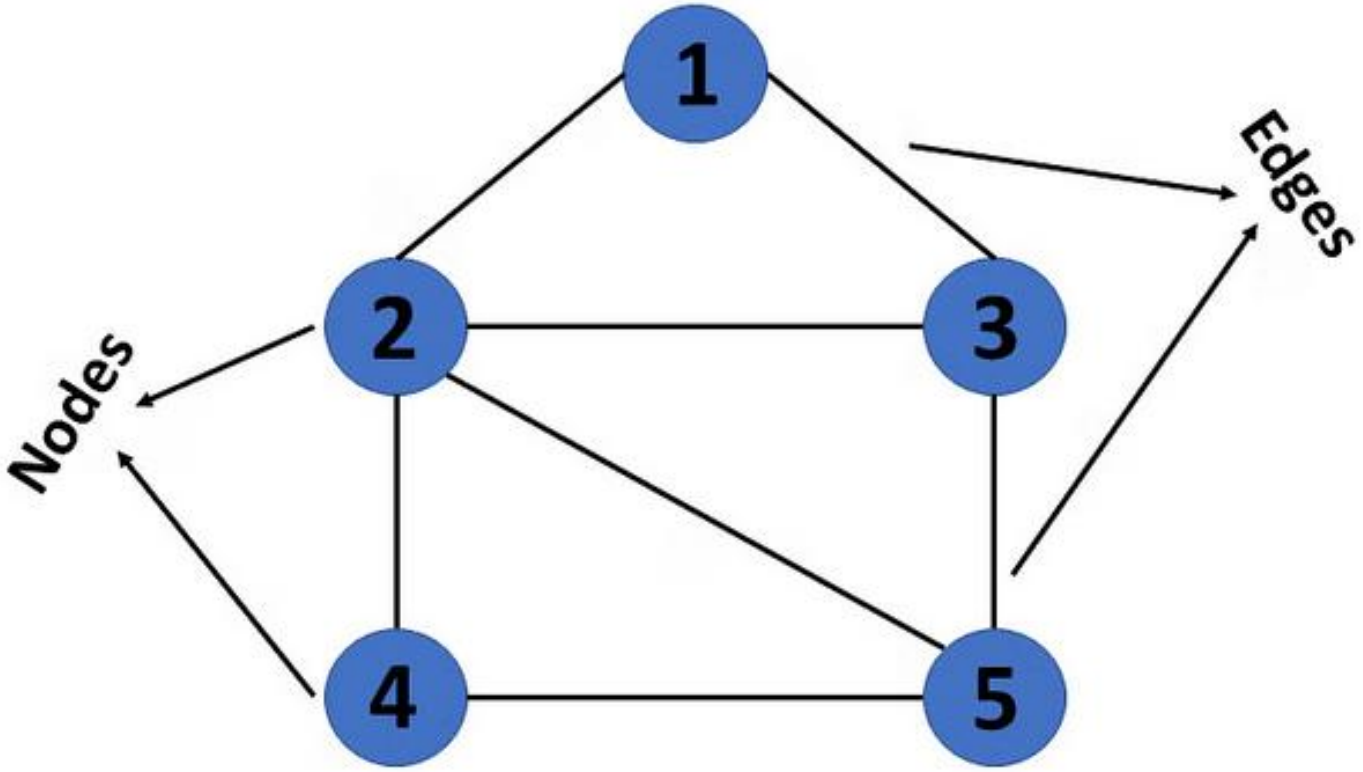


University of Szeged

Robert Nagy

2025/26. Autumn Semester

Graph



Some graph properties

Nodes (Vertices) – *V*

- Degree of a vertex

Edges – *E*

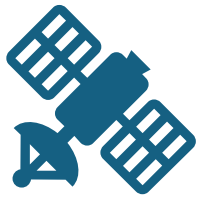
Paths

https://medium.com/@sarinyaswift/intro-to-the-graph-data-structure-a8277c6a2ad9

Graphs in real life



Social networks



Navigation systems



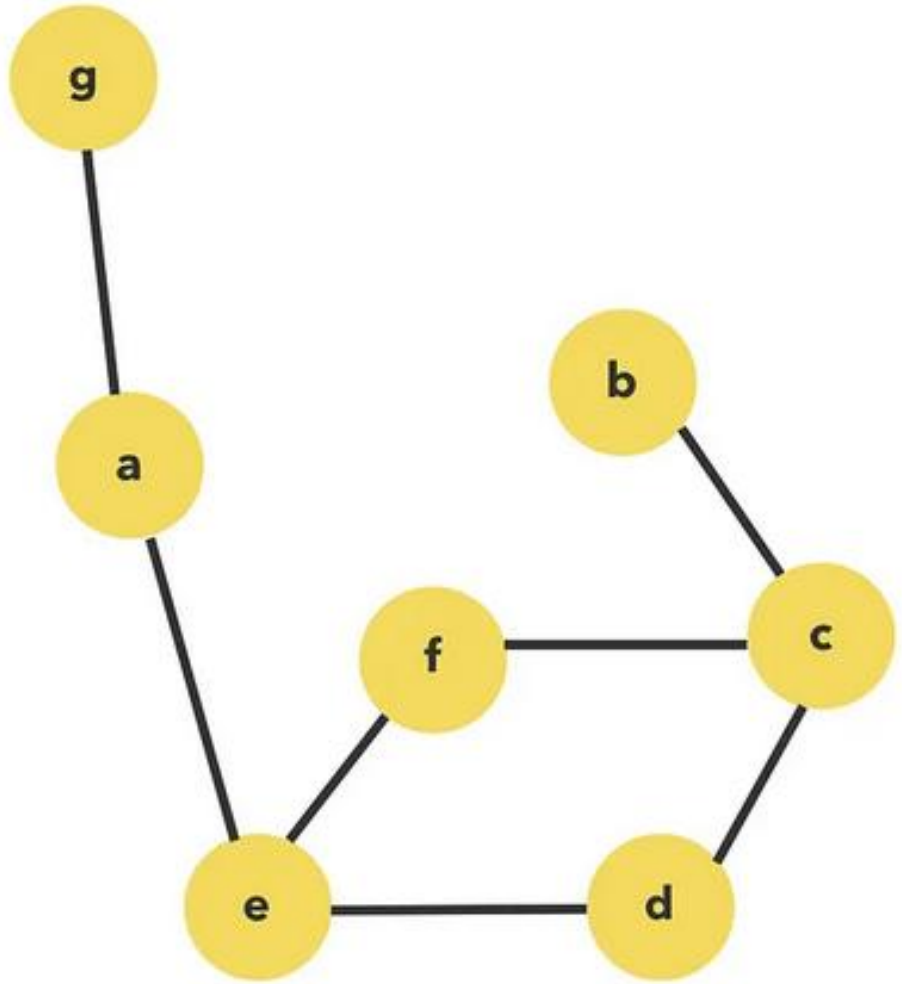
The internet



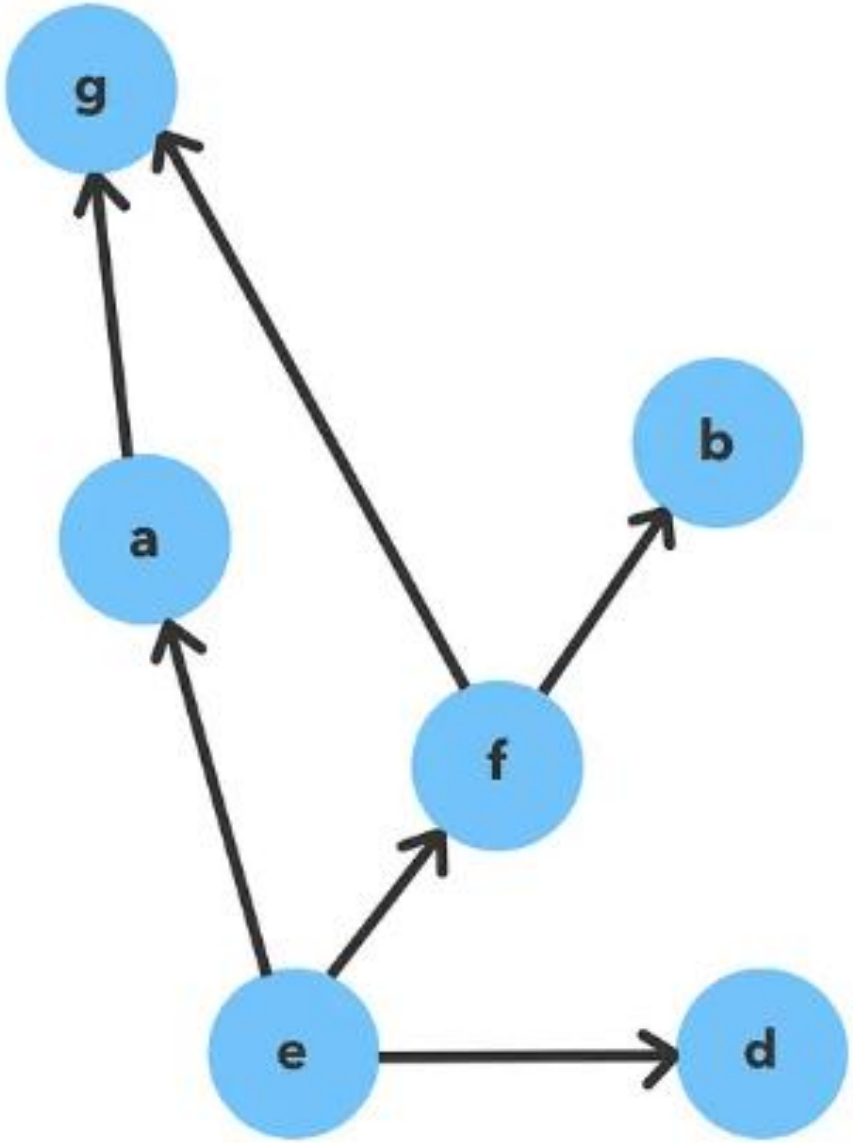
Search Engines

Different types of graphs

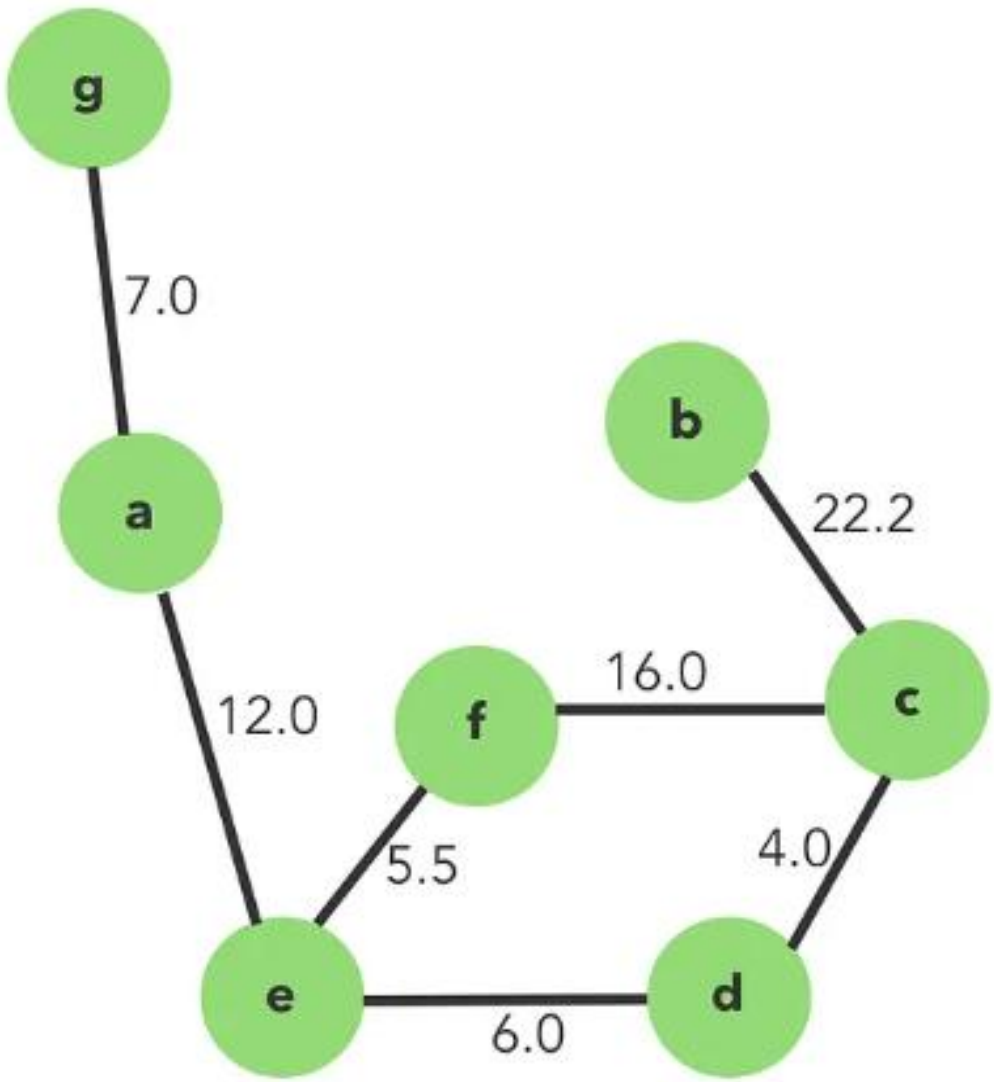
Undirected



Directed



Weighted

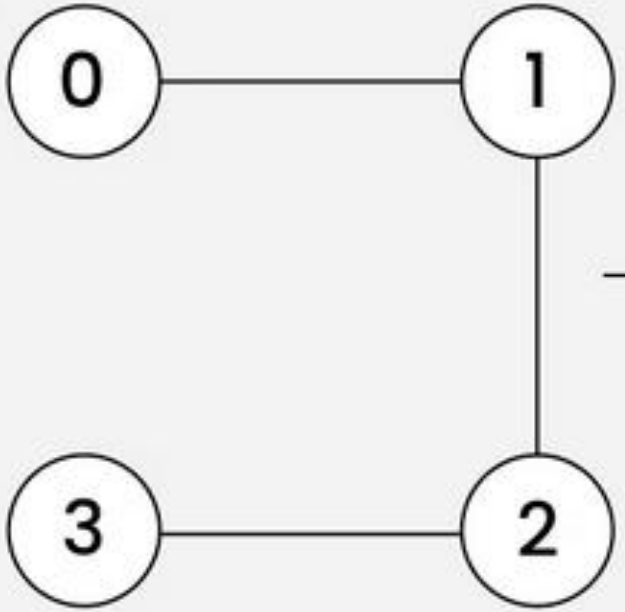


Graph representation

Adjacency matrix

Adjacency list

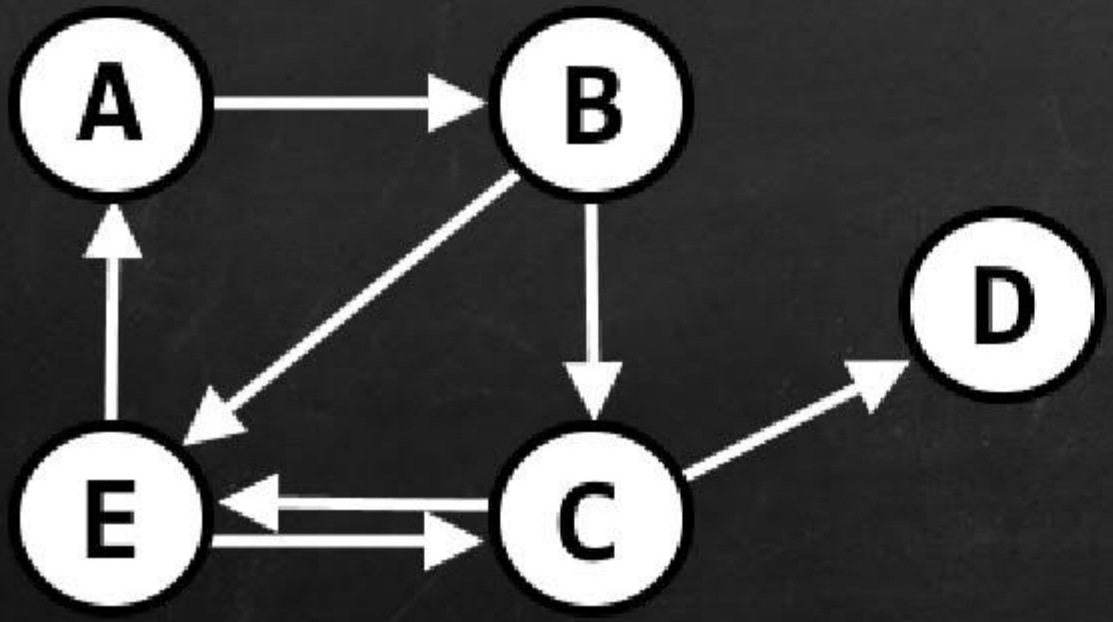
Adjacency Matrix for Undirected and Unweighted graph

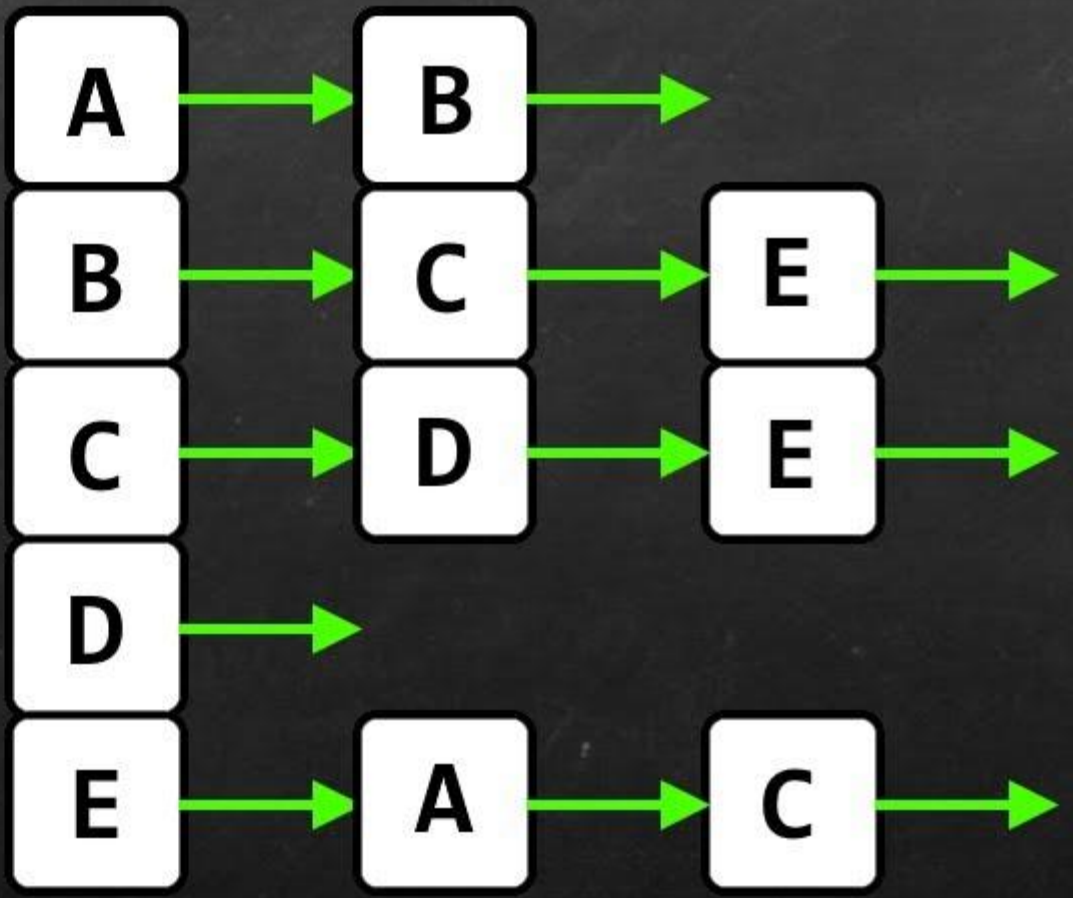


	0	1	2	3
0	0	1	0	0
1	1	0	1	0
2	0	1	0	1
3	0	0	1	0

adjacency list

https://www.youtube.com/watch?v=ee6zIj4J3-Y





Adjacency Matrix

- Good for dense graphs
- Edge checks take $O(1)$ time
- Space inefficient – $O(V^2)$

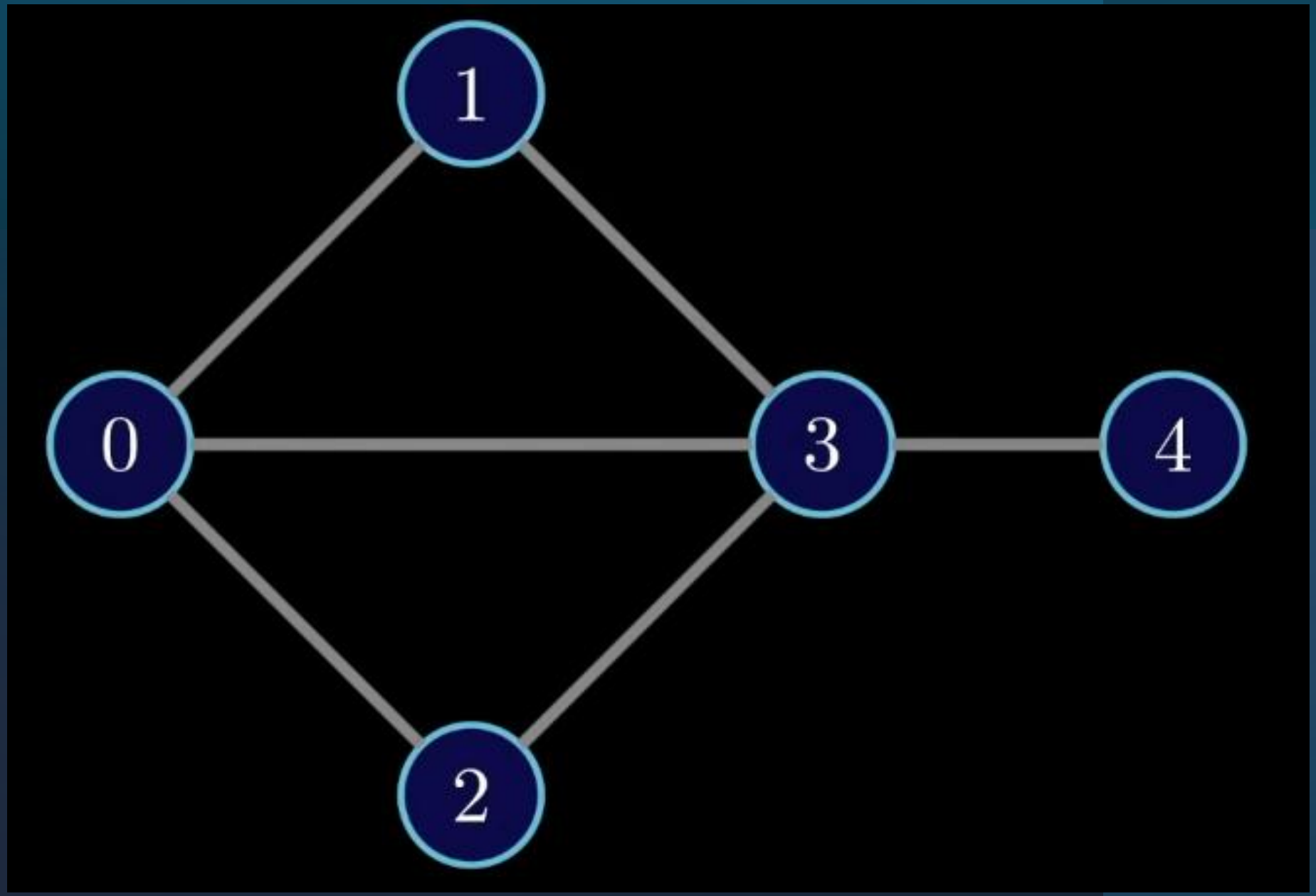
Adjacency List

- Space efficient – $O(V + E)$
- Checking neighbors is fast
- Better for sparse graphs

Graph Algorithms

BFS – Breadth First Search

https://www.youtube.com/watch?v=xlVX7dXLS64


1
0
3
4
2

```
1  visited = [False] * 5
2  graph = [[1,3,2],[0,3],[0,3],[0,1,2,4],[3]]
3  def BFS(graph, vertex):
4      global visited
5      queue = [vertex]
```

```
visited = [False] * 5
graph = [[1,3,2],[0,3],[0,3],[0,1,2,4],[3]]
def BFS(graph, vertex):
    global visited
    queue = [vertex]
    while len(queue):
        node = queue.pop(0)
        print(node)
        visited[node] = True
```

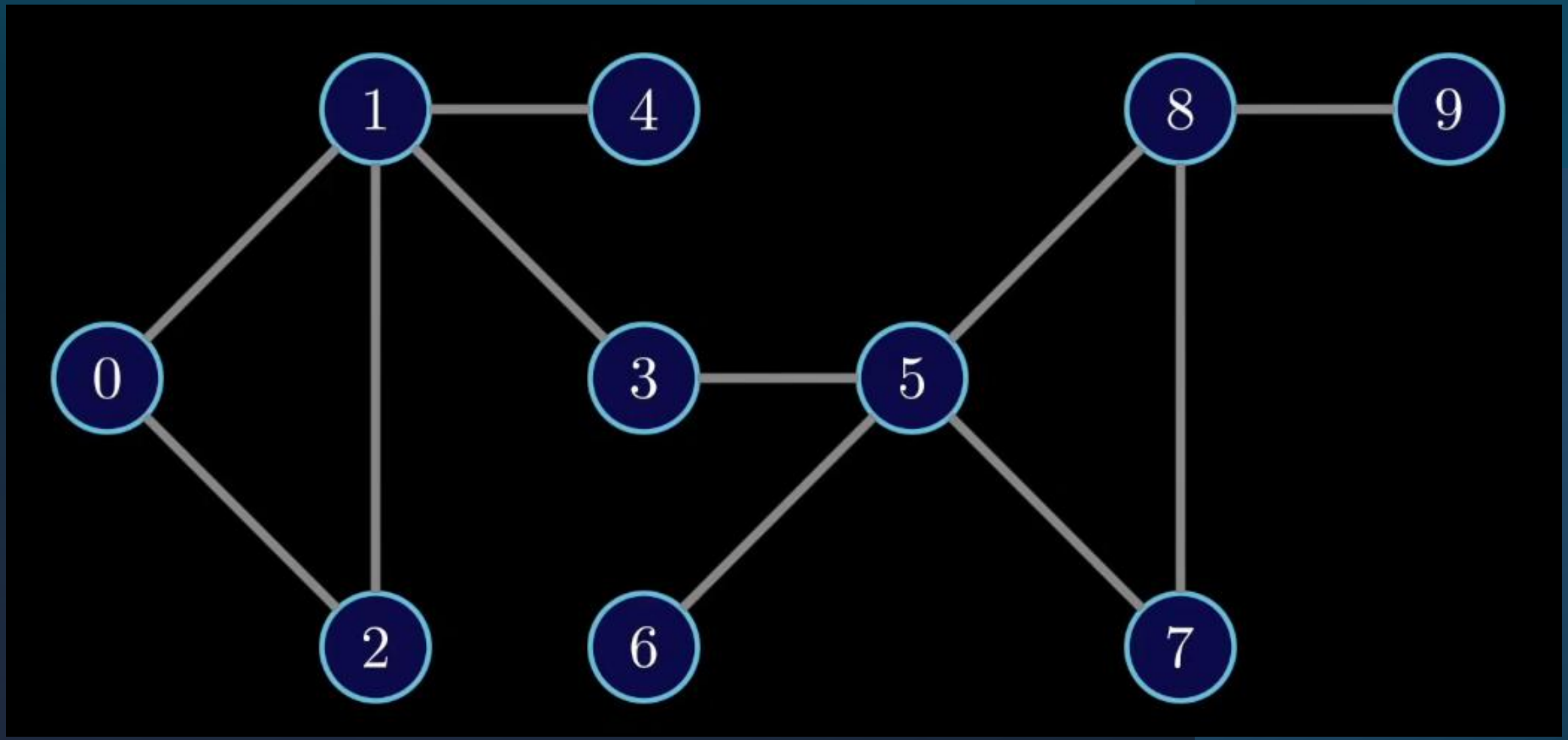
```
visited = [False] * 5
graph = [[1,3,2],[0,3],[0,3],[0,1,2,4],[3]]
def BFS(graph, vertex):
    global visited
    queue = [vertex]
    while len(queue):
        node = queue.pop(0)
        print(node)
        visited[node] = True
        for neighbor in graph[node]:
```

```
visited = [False] * 5
graph = [[1,3,2],[0,3],[0,3],[0,1,2,4],[3]]
def BFS(graph, vertex):
    global visited
    queue = [vertex]
    while len(queue):
        node = queue.pop(0)
        print(node)
        visited[node] = True
        for neighbor in graph[node]:
            if not visited[neighbor]:
                visited[neighbor] = True
                queue.append(neighbor)
```

```
1  visited = [False] * 5
2  graph = [[1,3,2],[0,3],[0,3],[0,1,2,4],[3]]
3  def BFS(graph, vertex):
4      global visited
5      queue = [vertex]
6      while len(queue):
7          node = queue.pop(0)
8          print(node)
9          visited[node] = True
10         for neighbor in graph[node]:
11             if not visited[neighbor]:
12                 visited[neighbor] = True
13                 queue.append(neighbor)
14
15 BFS(graph, 0)
```

DFS – Depth First Search

https://www.youtube.com/watch?v=PMMc4VsIacU


1
4
8
9
0
3
5
2
6
7

2 DFS Variations

Preorder

Postorder

Preorder

```
1  visited = [False] * 10
2  graph = [[1,2],[0,2,3,4],[0,1],[1,5],[1], \
3  [3,6,7,8],[5],[5,8],[5,7,9], [8]]
4  def DFS(graph, vertex):
5      global visited
```

```
visited = [False] * 10
graph = [[1,2],[0,2,3,4],[0,1],[1,5],[1], \
[3,6,7,8],[5],[5,8],[5,7,9], [8]]
def DFS(graph, vertex):
    global visited
    if visited[vertex]:
        return
    print(vertex)
    visited[vertex] = True
```

```
1  visited = [False] * 10
2  graph = [[1,2],[0,2,3,4],[0,1],[1,5],[1], \
3  [3,6,7,8],[5],[5,8],[5,7,9], [8]]
4  def DFS(graph, vertex):
5      global visited
6      if visited[vertex]:
7          return
8      print(vertex)
9      visited[vertex] = True
10     for neighbor in graph[vertex]:
11         DFS(graph, neighbor)
12
13 DFS(graph, 0)
```

```
1  visited = [False] * 10
2  graph = [[1,2],[2,3,4],[],[5],[],
3  [6,7,8],[],[8],[9], []]
4  def DFS(graph, vertex):
5      global visited
6      if visited[vertex]:
7          return
8      visited[vertex] = True
9      for neighbor in graph[vertex]:
10         DFS(graph, neighbor)
11     print(vertex)
12
```

Post-order

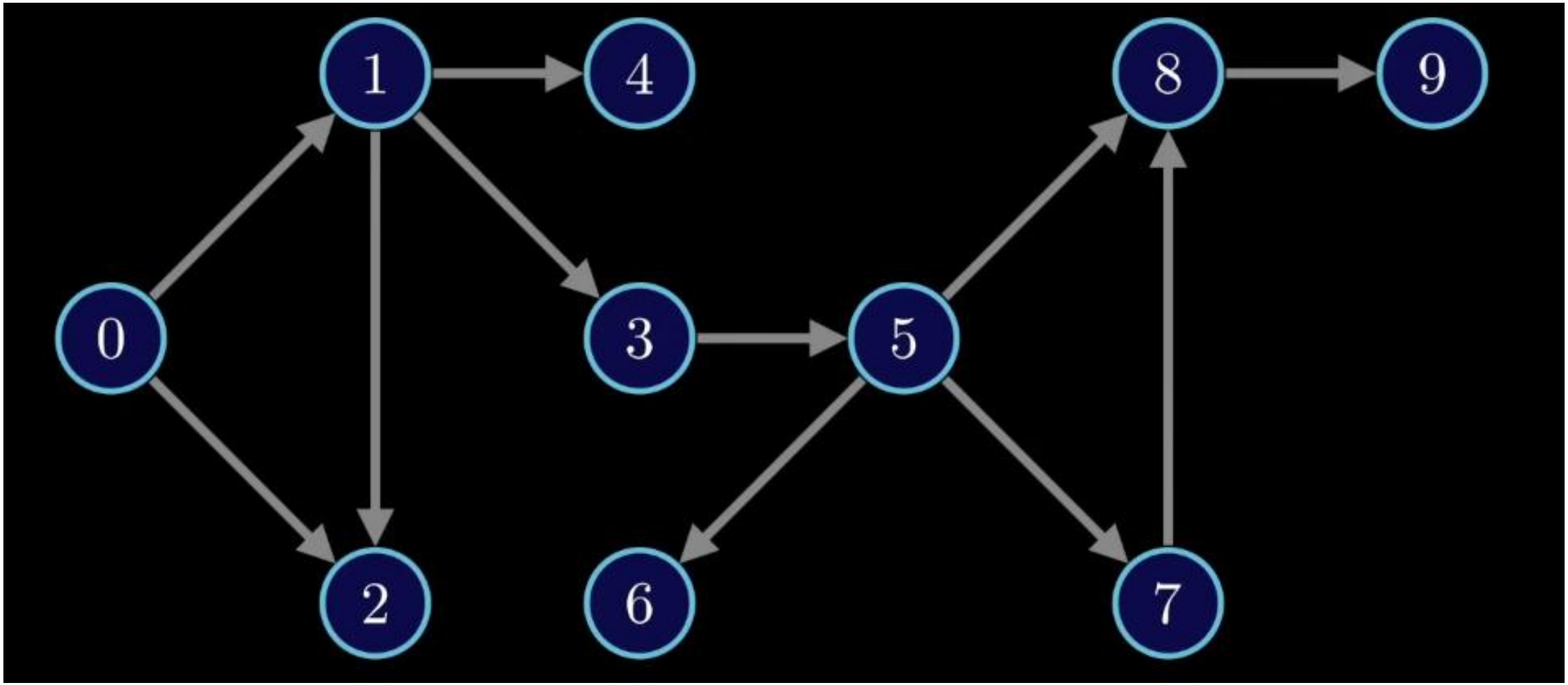
Topological sort

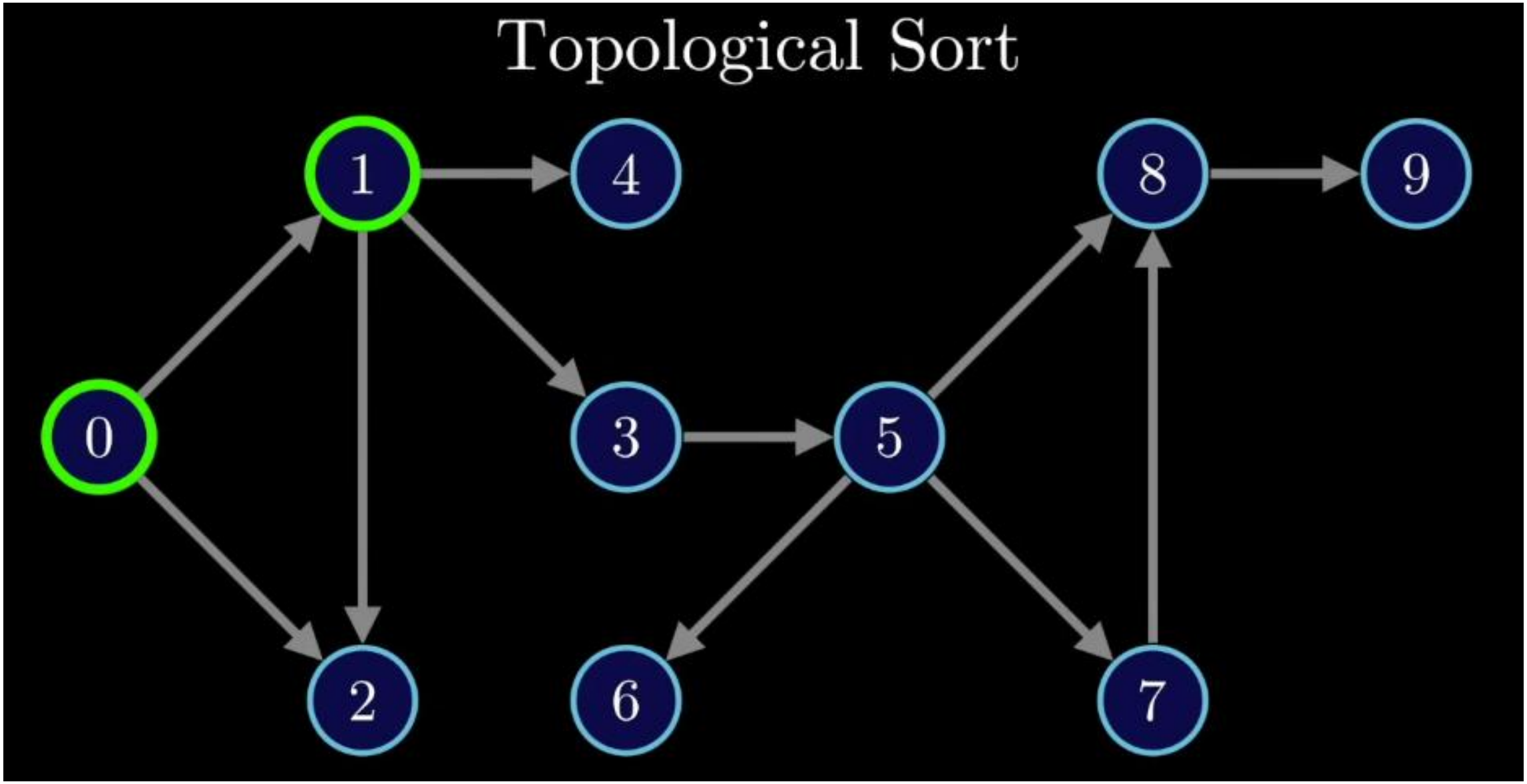
Directed, acyclic graphs

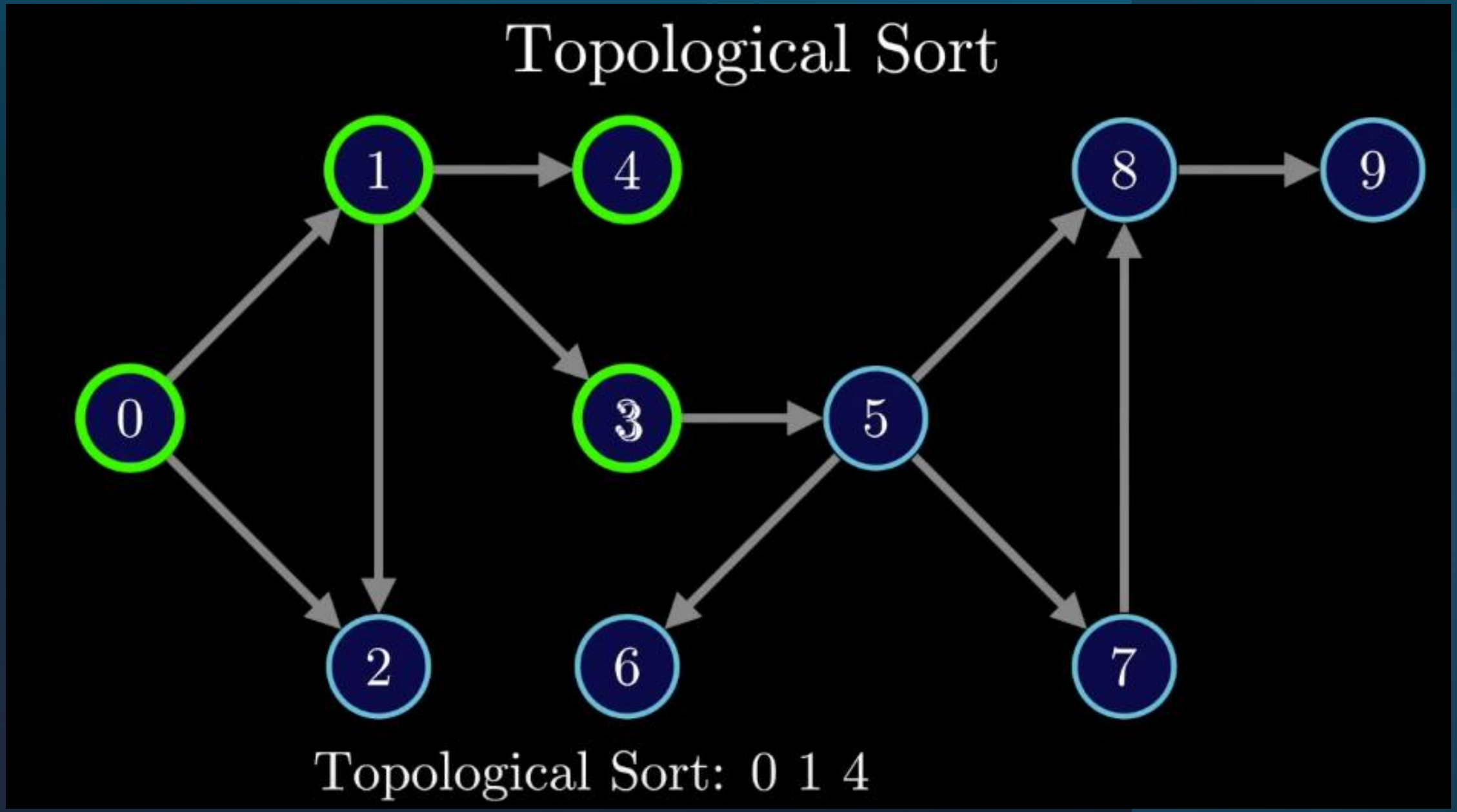
Meaning

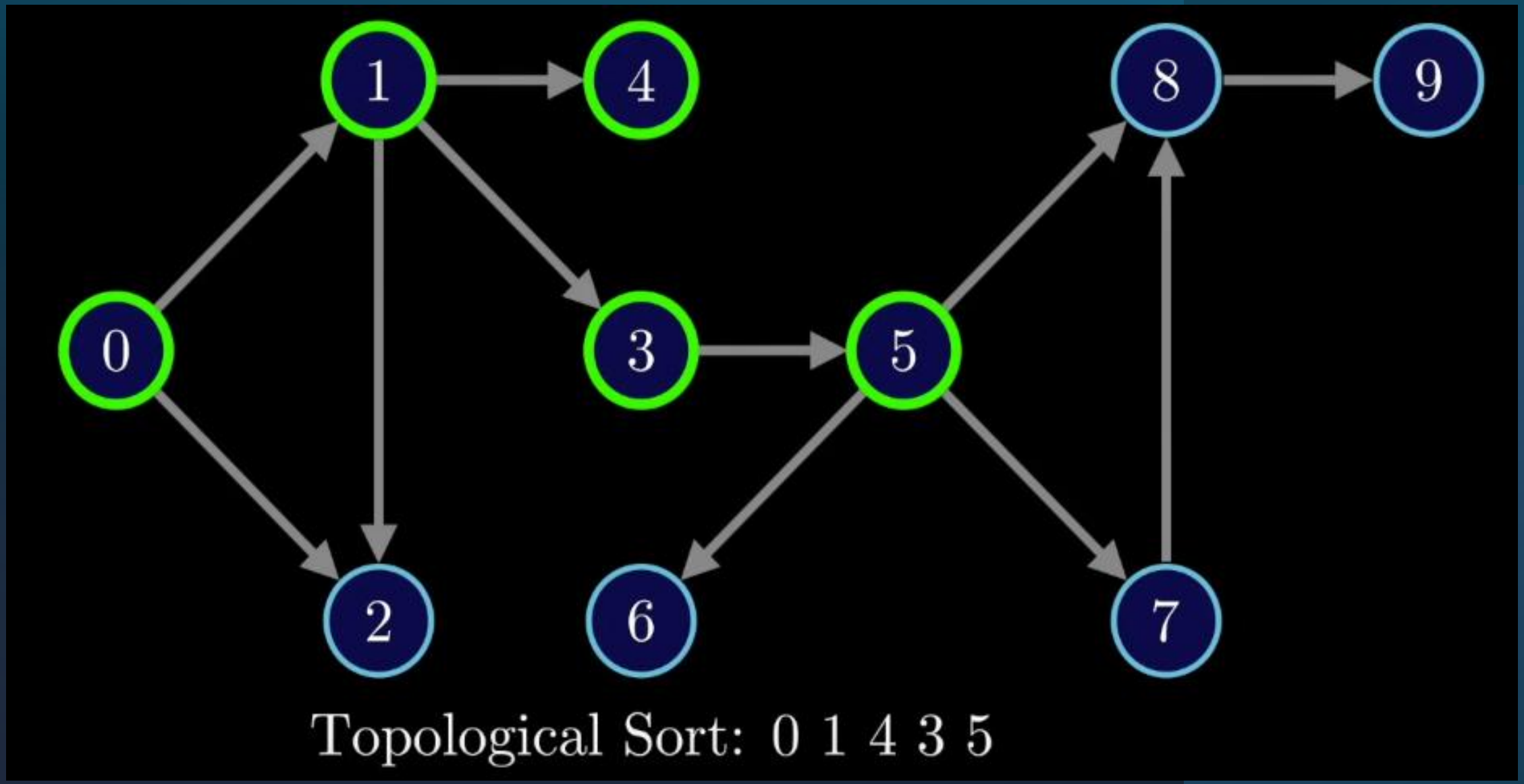
Think of nodes as tasks

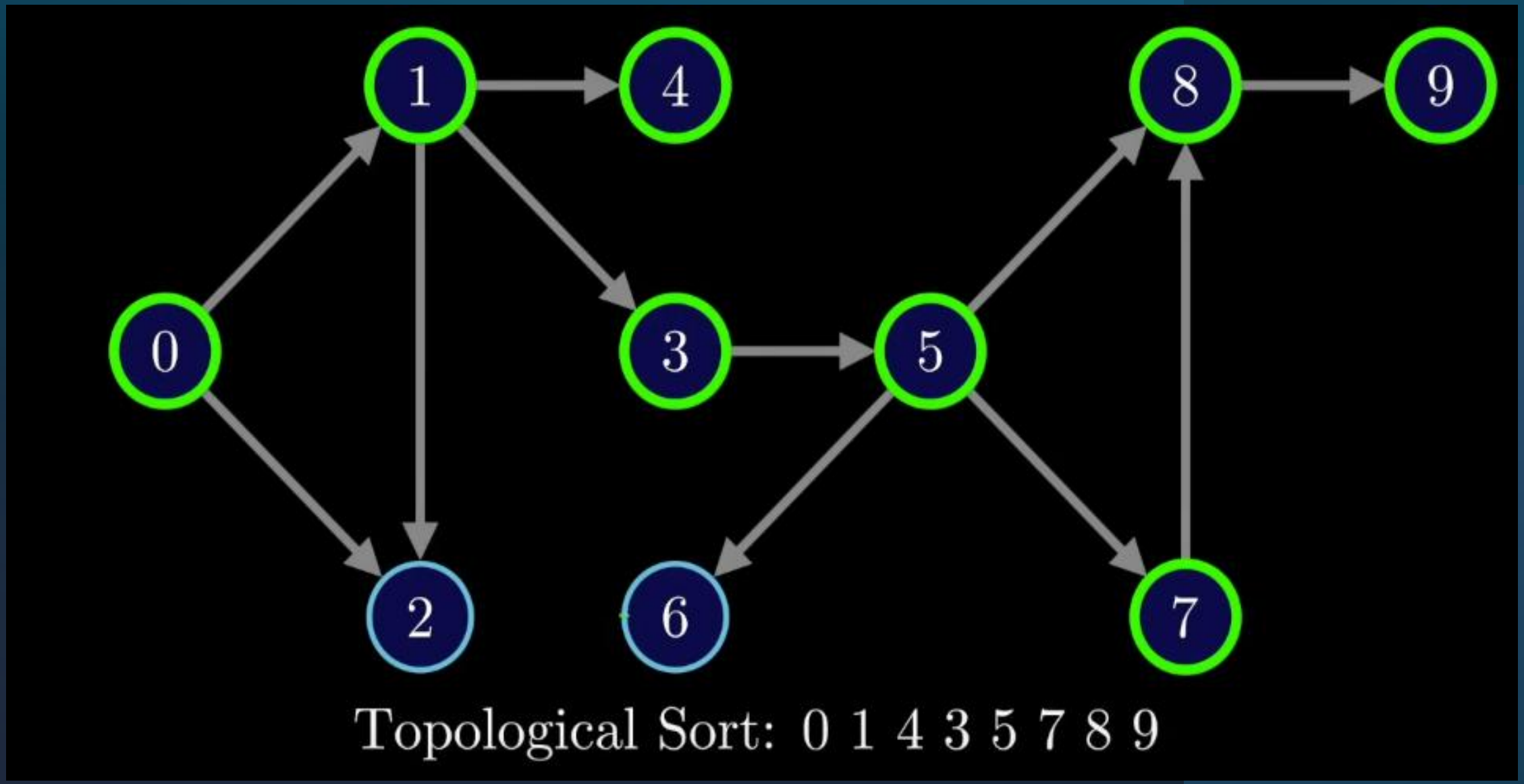
0 → 1 means task 0 must be done before task 1

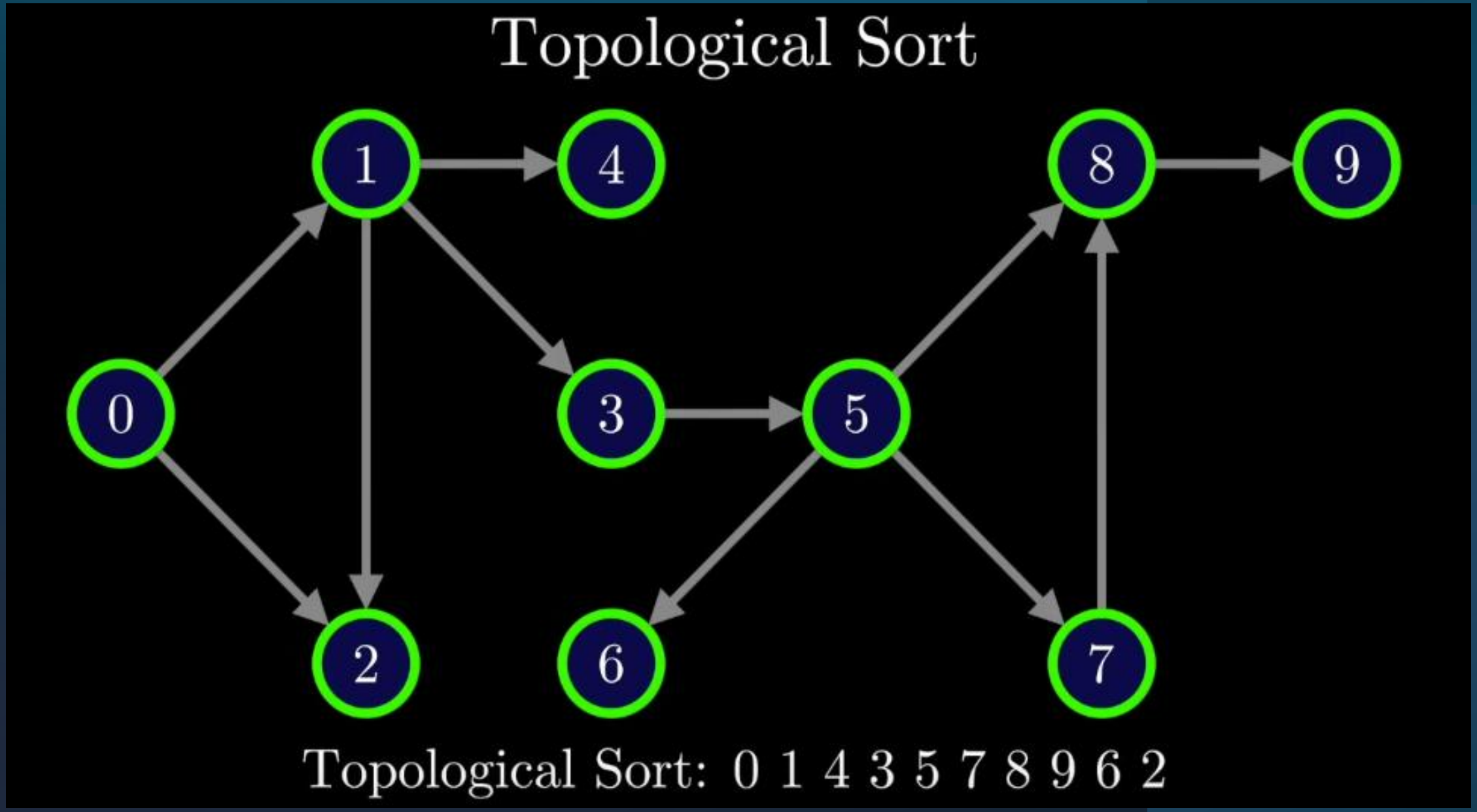

1
4
8
9
0
3
5
2
6
7


Topological Sort
1
4
8
9
0
3
5
2
6
7

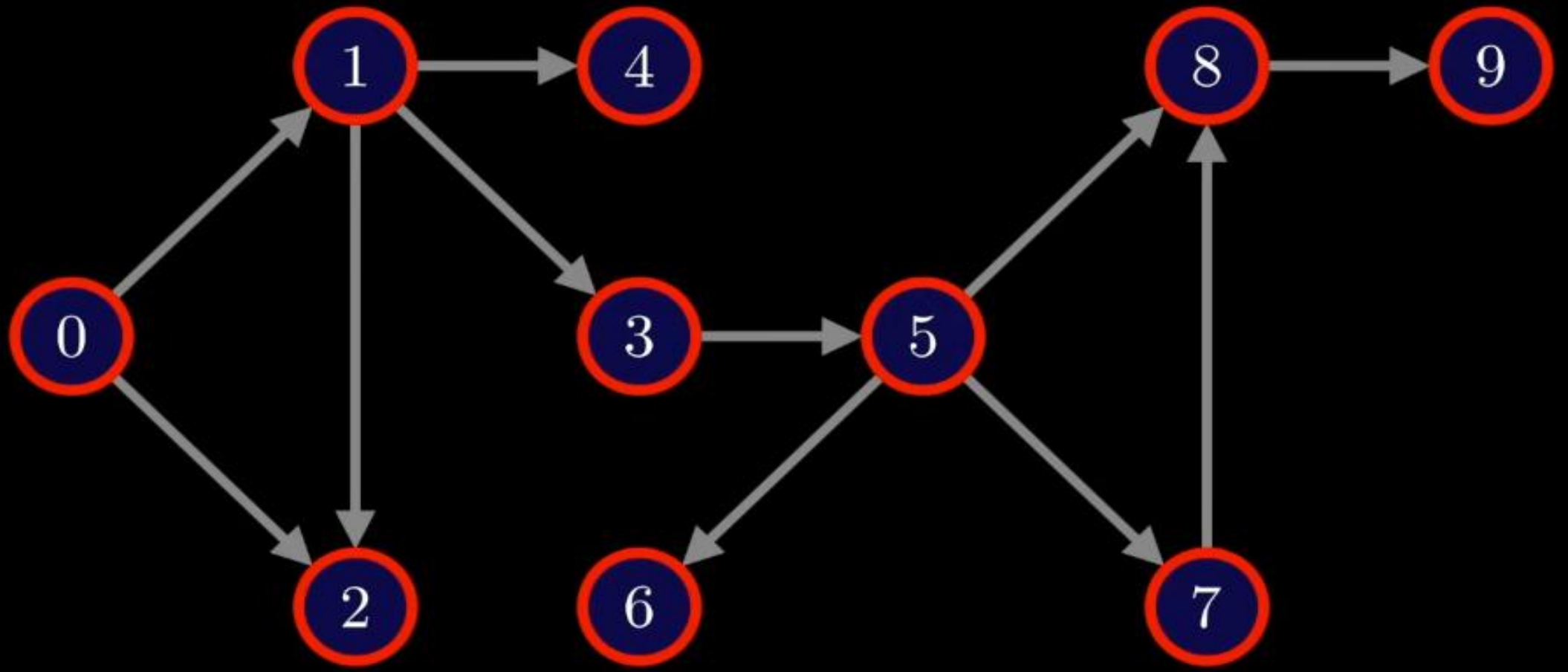

Topological Sort
0
1
2
3
4
5
6
7
8
9
Topological Sort: 0 1 4


1
4
8
9
0
3
5
2
6
7
Topological Sort: 0 1 4 3 5


1
4
8
9
0
3
5
2
6
7
Topological Sort: 0 1 4 3 5 7 8 9


Topological Sort
1
4
8
9
0
3
5
2
6
7
Topological Sort: 0 1 4 3 5 7 8 9 6 2

Topological Sort



Topological Sort: 0 1 4 3 5 7 8 9 6 2

DFS Postorder: 2 6 9 8 7 5 3 4 1 0

Reversing a post-order DFS always gives a valid topological sort.

F.Y.I.

```
visited = [False] * 10
graph = [[1,2],[0,2,3,4],[0,1],[1,5],[1],
[3,6,7,8],[5],[5,8],[5,7,9], [8]]
def DFS(graph, vertex):
    global visited
    stack = [vertex]
    while len(stack):
        node = stack.pop(0)
        print(node)
        visited[node] = True
        for neighbor in graph[node]:
            if not visited[neighbor]:
                visited[neighbor] = True
                stack.insert(0, neighbor)

DFS(graph, 0)
```